

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ  
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

**Отчёт**

**Лабораторная работа №2**

Подключение внешнего API средствами fetch. Моковое API на JSON Server

Выполнил:

Малороссиянцев Борис

Группа J3212

Проверил:

Добряков Д.И.

Санкт-Петербург, 2026 г.

## Задача

В рамках лабораторной работы требовалось расширить проект Wanderlust (сайт для планирования путешествий), разработанный в ЛР1, путём подключения к внешнему API средствами fetch. Конкретно требовалось:

- Реализовать моковое REST API на базе JSON Server и собственного серверного middleware (server.js) с поддержкой регистрации и входа пользователей.
- Реализовать авторизацию: токен на основе Base64-кодирования, сохранение сессии в localStorage, защита защищённых маршрутов.
- Перевести работу с заметками и сохранёнными маршрутами с localStorage на API-запросы.
- Подключить два открытых внешних API: OpenWeatherMap (текущая погода на странице направления) и REST Countries (справочная информация о стране).
- Вынести всю логику работы с API в отдельный модуль api.js, логику сессий — в auth.js.

## Структура проекта

По сравнению с ЛР1 в проект добавлены следующие файлы:

| Файл         | Тип        | Назначение                                                                                                  |
|--------------|------------|-------------------------------------------------------------------------------------------------------------|
| server.js    | Node.js    | Моковый REST-сервер на базе JSON Server с кастомными маршрутами /register и /login и middleware авторизации |
| db.json      | JSON       | База данных JSON Server: коллекции users, notes, savedRoutes, trips                                         |
| package.json | NPM        | Зависимости проекта: json-server 0.17.4                                                                     |
| js/api.js    | JavaScript | Модуль API: все fetch-запросы к серверу и внешним сервисам                                                  |
| js/auth.js   | JavaScript | Модуль Auth: управление сессией пользователя через localStorage                                             |
| js/app.js    | JavaScript | Основная логика приложения (расширена по сравнению с ЛР1)                                                   |

## Серверная часть (server.js)

Для имитации работы с API реализован сервер на Node.js с использованием библиотеки json-server версии 0.17.4. Сервер запускается командой npm start и слушает порт 3000.

## Регистрация и вход

Добавлены два кастомных маршрута, которые обрабатываются до передачи запроса стандартному роутеру JSON Server:

- POST /register — создаёт нового пользователя в коллекции users. Выполняет валидацию: наличие обязательных полей, минимальная длина пароля (8 символов), уникальность email. Возвращает токен и объект пользователя.
- POST /login — ищет пользователя по email и паролю. Возвращает токен и объект пользователя при совпадении, 401 при несовпадении.

## Токен авторизации

Токен формируется как Base64-кодирование строки вида id:email. Это упрощённый подход, имитирующий работу с JWT без подписи. Реализованы вспомогательные функции makeToken() и parseToken().

## Middleware защиты маршрутов

Все маршруты, кроме /register и /login, защищены проверкой токена. Middleware извлекает токен из заголовка Authorization: Bearer <token>, декодирует его и сохраняет данные пользователя в req.currentUser. При отсутствии или недействительном токене возвращается 401.

## Клиентский модуль API (js/api.js)

Весь код взаимодействия с сетью вынесен в IIFE-модуль API. Базовый адрес сервера задан константой BASE = 'http://localhost:3000'. Реализованы следующие функции:

| Функция              | Назначение                                                                 |
|----------------------|----------------------------------------------------------------------------|
| register()           | POST /register — регистрация нового пользователя                           |
| login()              | POST /login — вход, получение токена                                       |
| getNotes()           | GET /notes?userId=...&_sort=id&_order=desc — заметки текущего пользователя |
| createNote()         | POST /notes — создание заметки с привязкой к userId                        |
| deleteNote(id)       | DELETE /notes/:id — удаление заметки                                       |
| getSavedRoutes()     | GET /savedRoutes?userId=... — сохранённые маршруты                         |
| saveRoute()          | POST /savedRoutes — сохранение маршрута                                    |
| unsaveRoute()        | GET + DELETE /savedRoutes — удаление сохранённого маршрута                 |
| getTrips()           | GET /trips?userId=... — поездки пользователя                               |
| getWeather(city)     | GET api.openweathermap.org — текущая погода по городу                      |
| getCountryInfo(name) | GET restcountries.com — данные о стране                                    |

Все функции используют fetch с async/await. Обработка ошибок централизована в вспомогательной функции handleResponse(), которая разбирает тело ответа и бросает исключение при !res.ok. Заголовок Authorization добавляется автоматически функцией authHeaders(), получающей токен из localStorage через Auth.

## Модуль сессии (js/auth.js)

Модуль Auth управляет пользовательской сессией и реализован как IIFE, возвращающий публичный API. Данные сессии хранятся в localStorage под ключами wl\_token и wl\_user.

Ключевые методы модуля:

- saveSession(token, user) — сохраняет токен и данные пользователя после успешного входа или регистрации.
- clearSession() / logout() — очищает localStorage и перенаправляет на login.html.
- isLoggedIn() — проверяет наличие токена.
- requireAuth() — перенаправляет на страницу входа, если сессии нет; вызывается в начале каждой защищённой страницы.
- requireGuest() — перенаправляет на dashboard.html, если пользователь уже авторизован; вызывается на страницах входа и регистрации.
- getUser(), getUserId(), getFullName(), getInitials() — вспомогательные методы для отображения данных пользователя в UI.

## Авторизация (login.html, register.html)

Формы входа и регистрации полностью переведены на работу с API. Атрибут action="dashboard.html" из ЛР1 удалён — отправка форм перехвачена обработчиком submit с preventDefault().

Страница входа:

- При открытии вызывается Auth.requireGuest() — авторизованный пользователь немедленно перенаправляется в кабинет.
- При отправке формы выполняется await API.login(email, password).
- При успехе: Auth.saveSession(), toast-уведомление, редирект на dashboard.html через 500 мс.
- При ошибке: выводится сообщение из ответа сервера (err.error) через showToast.
- Кнопка «Войти» переходит в состояние загрузки (setLoading) на время запроса.

Страница регистрации:

- Клиентская валидация дублирует серверную: проверяются обязательные поля и длина пароля.
- При успехе выполняется та же цепочка: saveSession → toast → редирект.
- Предзаполненные тестовые данные удалены из полей формы.

## Личный кабинет (dashboard.html)

Страница защищена вызовом Auth.requireAuth() в initDashboardPage(). Данные пользователя (имя, инициалы) берутся из Auth.getFullName() и Auth.getInitials() и подставляются в UI функцией fillUserUI().

## Заметки

Работа с заметками полностью переведена с localStorage на API:

- При загрузке страницы вызывается loadNotesFromAPI() — GET /notes с фильтром по userId и сортировкой по убыванию id. На время загрузки отображается spinner.
- Добавление заметки: POST /notes через API.createNote(). После успешного ответа обновляется список.
- Удаление заметки: DELETE /notes/:id через API.deleteNote(id). Список перерисовывается после подтверждения.
- Если заметок нет, отображается демонстрационный блок (DEMO\_NOTES\_HTML) для наглядности интерфейса.

## Сохранённые маршруты

На странице поиска кнопка «Сохранить» вызывает API.saveRoute(destinationId) или API.unsaveRoute(destinationId) в зависимости от текущего состояния. Состояние кнопки обновляется немедленно (оптимистичный UI), а запрос к API выполняется асинхронно в фоне.

## Страница направления (destination.html)

Страница переработана для динамической загрузки данных в зависимости от выбранного направления. При открытии из URL-параметра ?id=paris (или другого id) извлекается нужный объект из массива DESTINATIONS.

## OpenWeatherMap API

В правой колонке страницы отображается блок текущей погоды. Функция loadWeather(city) выполняет GET-запрос к <https://api.openweathermap.org/data/2.5/weather> с параметрами:

- q — английское название города из поля weatherCity объекта направления;
- appid — API-ключ;
- units=metric — температура в градусах Цельсия;
- lang=ru — описание погоды на русском языке.

В блоке отображаются: иконка погоды с сервера OpenWeatherMap, температура с учётом знака, текстовое описание, ощущаемая температура, влажность и скорость ветра. На время загрузки выводится spinner, при ошибке — лаконичное сообщение «Погода временно недоступна».

## REST Countries API

Параллельно с погодой загружается информация о стране через функцию loadCountryInfo(countryName). Запрос направляется к <https://restcountries.com/v3.1/name/:name> с фильтром полей (fields). В блоке отображаются: столица, регион, население (в миллионах), валюта и языки.

Оба внешних API-запроса выполняются независимо; ошибка одного не влияет на другой.

## Интеграция с внешними API

Каждому направлению в массиве DESTINATIONS добавлены два новых поля:

- `weatherCity` — английское название города для запроса к OpenWeatherMap (например: 'Paris', 'Male', 'Punta Arenas').
- `countryEn` — английское название страны для запроса к REST Countries (например: 'France', 'Maldives', 'Chile').

Данные внешних API не кэшируются — при каждом открытии страницы направления выполняются свежие запросы. Это соответствует требованию «работы с внешним API» без добавления избыточной сложности.

## Изменения в `app.js`

Файл расширен с 332 строк (ЛР1) до 772 строк. Основные изменения:

- Добавлена функция `fillUserUI()` — подставляет имя, инициалы и выход пользователя на все защищённые страницы.
- Добавлена функция `setLoading(btn, state)` — переводит кнопку в состояние ожидания (spinner + disabled) при асинхронных операциях.
- `initLoginPage()` и `initRegisterPage()` полностью переписаны — теперь используют `async/await` и API-модуль вместо HTML-атрибутов.
- `initDashboardPage()` переведён на API: заметки загружаются с сервера, добавляются и удаляются через `fetch`.
- `initDestinationPage()` принимает параметр `id` из URL, динамически наполняет страницу данными нужного направления и запускает `loadWeather()` и `loadCountryInfo()`.
- В данные каждого направления (DESTINATIONS) добавлены поля `weatherCity` и `countryEn` для работы с внешними API.
- Добавлены богатые статические данные для каждого направления: подробные достопримечательности, советы по еде и транспорту, отзывы и данные для карты.

## База данных JSON Server (`db.json`)

Файл `db.json` определяет четыре коллекции, которые JSON Server превращает в REST-эндпоинты:

| Коллекция                | Содержимое                                                                                                                                           |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>users</code>       | Пользователи. Изначально пустой массив; заполняется через <code>/register</code>                                                                     |
| <code>notes</code>       | Путевые заметки. Хранит поля: <code>id</code> , <code>userId</code> , <code>title</code> , <code>text</code> , <code>dest</code> , <code>date</code> |
| <code>savedRoutes</code> | Сохранившиеся маршруты. Хранит: <code>id</code> , <code>userId</code> , <code>destinationId</code>                                                   |

|       |                                                                                            |
|-------|--------------------------------------------------------------------------------------------|
| trips | Совместные поездки. Включает демо-запись «Мальдивы 2025» с задачами и остановками маршрута |
|-------|--------------------------------------------------------------------------------------------|

## Вывод

В ходе выполнения лабораторной работы проект Wanderlust был расширен до полноценного клиент-серверного приложения с использованием fetch API.

Реализовано моковое REST API на базе JSON Server с кастомным Node.js middleware, обеспечивающим регистрацию, вход и защиту ресурсов через токен-авторизацию. Клиентская логика разделена на три модуля: app.js (UI и бизнес-логика), api.js (сетевой слой) и auth.js (управление сессией).

Дополнительно подключены два открытых внешних API: OpenWeatherMap для отображения актуальной погоды в выбранном направлении и REST Countries для справочной информации о стране. Оба запроса выполняются асинхронно с обработкой состояний загрузки и ошибок.

Выполнение лабораторной работы позволило получить практические навыки работы с fetch/async/await, проектирования REST API на JSON Server, организации авторизации на фронтенде и интеграции внешних API в многостраничное веб-приложение.